Pixels-to-Predictions: SmolVLM-500M QLoRA Fine-Tuning for ScienceQA

Yikai Sun

NYU Tandon School of Engineering

ys4035@nyu.edu

Abstract

We fine-tune `HuggingFaceTB/SmolVLM-500M-Instruct` for the *Pixels-to-Predictions* multimodal multiple-choice science QA competition under a strict 5,000,000-trainable-parameter budget and an offline-evaluation rule. We apply QLoRA (4-bit NF4 + double quantization) with rank-8 LoRA across all seven text-decoder linear projections, yielding 4,341,760 trainable parameters (87% of the cap). We score by single-token log-likelihood at the answer-letter position and apply a position-bias-aware test-time augmentation (smart-TTA) that averages logits over $K = 4$ random choice permutations while bypassing TTA on 2-choice questions where it empirically degrades accuracy. Our best configuration ($r = 8$, 384 px, 5 epochs) achieves **0.81287** on the public leaderboard, $+4.83$ pp over the same model with standard scoring (0.76458). Five ablations isolate which axes of capacity matter at this scale: training duration is the dominant lever ($+3.6$ pp from 3 to 5 epochs), but extending to 7 epochs *reduces* LB by 1.0 pp (overfitting), establishing 5 epochs as near-optimal; doubling LoRA rank from 4 to 8 contributes $+1.8$ pp at matched training duration; higher visual resolution (384 → 512 px) is statistically tied with baseline.

1 Introduction

The competition task is multimodal multiple-choice science QA: each example pairs an image (diagrams, maps, charts, photos) with a question, 2–5 textual answers, and optional `hint` / `lecture` context. The dataset matches the ScienceQA schema.

The dataset's `num_choices` distribution is approximately 21 / 50 / 25 / 4% over $\{2, 3, 4, 5\}$, giving a random-guess baseline of ~ 32.5%, *not* 25%. The base SmolVLM-500M, used zero-shot, achieves **41.5%** on a 200-example val sample (binomial SE ≈ 3.5 pp), with a strong "A" bias (71.5% of predictions choose the first option) and 5-choice accuracy below random (18.6%) — we hypothesize because the letter "E" rarely appears as a multiple-choice option in pretraining text. These two patterns motivate the loss-masked SFT and choice-permutation TTA below.

Contributions.

1. A QLoRA fine-tuning pipeline targeting all seven text-decoder linear projections at rank 8, sized to 87% of the 5M trainable-param budget.
2. A position-bias-aware smart-TTA at inference that averages over $K = 4$ random choice permutations on 3+ choice questions while bypassing TTA on 2-choice questions where it degrades accuracy.
3. A data-curve-driven ablation methodology: each ablation tested a hypothesis derived from the previous run's loss curve or val/LB pattern, with the headline finding that training duration peaks at 5 epochs and overfits at 7.
4. Diagnosis and resolution of a QLoRA loss-explosion failure on T4 (paged AdamW 8-bit, aggressive grad clipping at 0.3, extended warmup ratio 0.1), without which our first run diverged catastrophically.

Code: https://github.com/Palomarr/pixels-to-predictions.

2 Methodology

2.1 Architecture and Parameter Budget

`SmolVLM-500M-Instruct` consists of a SigLIP-Base vision encoder (~ 93M params, frozen),

a small modality projector (frozen), and a SmolLM2-360M text decoder (32 layers, hidden 960, 15 attention heads, 5 KV heads, MLP intermediate 2,560). We adapt only the text decoder via LoRA on all seven linear projections per layer (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`). At $r = 8$, $\alpha = 16$ ($\alpha/r = 2$), this gives **4,341,760 trainable parameters** (87% of the 5M cap). Three alternatives were considered (Table 1).

Allocation	r	Trainable	Notes
Attention-only (q, k, v, o)	16	3.28M	Wastes MLP capacity
Attention-only (q, v)	32	3.28M	Even narrower
MLP-only (g, u, d)	8	2.70M	Misses attention adaptation
All seven, $r = 8$	**8**	**4.34M**	**Selected**

Table 1: LoRA allocations under the 5M cap. The QLoRA paper finds low-rank adaptation across all linear layers beats higher-rank attention-only at matched budgets.

A regex `target_modules` is used to constrain LoRA to text-decoder layers only — naively passing `target_modules=[q_proj, …]` would also match the vision encoder's same-named projections, silently wasting ~ 30% of the budget on adapting frozen vision features.

2.2 Quantization: QLoRA

We load the base in 4-bit NF4 with double quantization, reducing base-model VRAM from ~ 1.0 GB (bf16) to ~ 250 MB (NF4). Compute dtype is nominally `bf16`, but T4 (sm_75) has no native bf16 tensor cores and these ops fall back to emulated kernels — a throughput penalty we did not separately measure against a fp16 alternative. The submission notebook runs offline and lacks `bitsandbytes`, so it loads the base in `bf16`; we discuss the resulting precision-mismatch gap in Section 6.1.

2.3 Prompt Format and Loss Masking

We follow the competition starter notebook's template:

```
<image>
Context:
{lecture, truncated to 800 chars}
{hint}

Question: {question}
Choices:
  A. {choice_0}
  B. {choice_1}
  ...
Answer: {letter — only at training}
```

The `<image>` token marks where the processor splices in visual tokens. We force `do_image_splitting=False` and `longest_edge=384` for exactly 81 visual tokens per image; the processor's default multi-tile splitting up to 1,536 px (~ 1,088 visual tokens) would blow past our 1,024-token sequence limit and waste compute on most ScienceQA images, which are < 512 px natively.

Loss masking. During SFT we set `labels` to -100 at every token position *except* the trailing answer-letter token, concentrating 100% of the gradient signal on "predict the right letter" rather than on language-modeling the lecture/question/hint context. Without this, our 4.34M-parameter budget would be largely consumed by a training objective that has no test-time analogue.

2.4 Inference: Log-Likelihood Scoring

For each choice $j \in \{0, ..., \text{num_choices} - 1\}$, we compute $\text{score}[j] = \log P\big(\text{letter}_j \mid \text{image}, \text{prompt}\big)$ from the next-token logits at the answer position, where letter_j is the single token for the j-th letter (`" A"`, `" B"`, …; the leading space matters under SmolLM2's BPE). Letters beyond `num_choices` are masked to $-\infty$ before argmax. One forward pass per example, fully deterministic — avoiding the brittleness of generation parsing (variable formats, hallucinated letters).

2.5 Test-Time Augmentation: Smart Choice-Permutation

The pretrained-baseline analysis (above) revealed strong position bias — the model prefers "A"

independent of choice content. After fine-tuning, residual position bias persists and grows with the number of letter positions involved.

For each test example we run $K = 4$ forward passes. Pass 0 uses the original choice ordering (so $K = 1$ matches the no-TTA path). Passes 1–3 each shuffle the choices into a random ordering and rescore. To average logits in *original-choice space*, we map each pass's letter-position logit back to the original choice it represents:

```
accum[i, original_choice] +=
letter_logits[i, shuffled_position]
```

After K passes, $\arg\max_i$ over `accum[i]` selects the highest-averaged-logprob original choice. Position bias cancels because each original choice now appears in every position roughly evenly.

Smart TTA: skip 2-choice. On val, $K = 4$ TTA hurts 2-choice questions by -1.23 pp while helping 3-, 4-, 5-choice by $+1.18$, $+2.78$, $+4.55$ pp. With only two unique permutations available at $n = 2$, $K = 4$ random sampling adds variance without canceling bias (which is small at $n = 2$ anyway). A `tta_skip_below=3` parameter bypasses TTA below the threshold, falling back to the identity-pass prediction. Smart-TTA yields $+1.91$ pp val and $+1.21$ pp LB on v1 — both larger than naive always-TTA.

2.6 Training Stability Fixes

Our first training run failed catastrophically: at step 20 the loss spiked to 2.66×10^{16}, and from steps 40–100 reported `0.000000` (gradient underflow). The pattern — 16 steps of stable forward+backward (one gradient-accumulation cycle) followed by a single divergent optimizer update — pointed to three independent fp16 issues that compound under QLoRA.

Fix	What it addresses
`optim="paged_adamw_8bit"`	Default `adamw_torch` keeps optimizer states in fp32; under QLoRA + bnb mixed-precision dequant, scale mismatches amplify. The 8-bit paged optimizer is the reference QLoRA recipe.
`max_grad_norm=0.3` (was 1.0)	Default 1.0 is calibrated for full fine-tuning. With only 4.34M params receiving gradients flowing through 511M frozen-quantized params, per-parameter step sizes are effectively much higher; aggressive clipping is the standard QLoRA practice.
`warmup_ratio=0.1` (was 0.03)	At 0.03, only ~ 12 warmup steps preceded peak LR=2e-4; the first optimizer update fired at step 16 with LR essentially at peak. 0.1 gives ~ 58 warmup steps, so by step 16 LR is ~ 40% of peak.

Table 2: QLoRA stability fixes that turned a divergent first run into reproducible training.

After applying all three fixes, training was stable across all subsequent runs.

3 Experimental Setup

Data splits. Train 3,109; val 1,048; test 1,008 (labels held out). At effective batch 16 and 3 epochs, only ~ 580 optimizer steps.

Optimizer & schedule. Paged AdamW 8-bit, learning rate 2×10^{-4}, cosine decay with 10% warmup, weight decay 0.0, max grad norm 0.3. Effective batch 16 (per-device 1 $\times$ gradient accumulation 16). Mixed precision: nominal `bf16` (T4 fallback, see Section 6.1).

Other. `max_seq_length=1024`, lectures pre-truncated to 800 characters, images forced to a single 384×384 tile (~ 81 visual tokens). Single random seed (42). NVIDIA T4 (16 GB) on Kaggle Free Tier; 3-epoch run ~ 2:44, 5-epoch ~ 4:50, 7-epoch ~ 6:30.

4 Main Result

Our best submission applies the v3 adapter ($r = 8$, 384 px, 5 epochs) with smart-TTA at $K = 4$:

Configuration	Public LB
Random baseline (analytic)	~ 0.325
Pretrained, no fine-tune (200-ex val proxy)	~ 0.415
v1 fine-tuned, no TTA	0.76458
v1 fine-tuned + smart-TTA	0.77665
v2 (img 512) + smart-TTA	0.77867
v5 ($r = 4$, 5 epochs) + smart-TTA	0.79476
v4 (7 epochs) + smart-TTA	0.80281
v3 ($r = 8$, 5 epochs) + smart-TTA	**0.81287**

Table 3: Public leaderboard scores. The pretrained-baseline row is a 200-example val proxy and not directly comparable to the LB rows (1,008-example hidden test); it anchors the qualitative gap from no-fine-tune to fine-tuned.

Net lift from training-side improvements (v1 → v3): +4.83 pp. Net lift from inference-only smart-TTA on v1: +1.21 pp.

5 Ablations

We vary training duration, LoRA rank, image resolution, and TTA strategy. All runs use a single random seed; with $n = 1\{,\}008$ test examples and accuracies in 0.77–0.81, the binomial standard error of any single-run LB estimate is ≈ 1.3 pp, so we treat deltas < 2 pp as suggestive rather than significant.

5.1 Headline Ablations Table

ID	Config	Train-val	LB
v1	$r = 8$, all-7, 384 px, 3 epochs (baseline)	0.7634	0.77665
v2	1. img_size=512	0.7586	0.77867
v3	**+ epochs=5 (best)**	**0.8015**	**0.81287**
v4	1. epochs=7	0.7996	0.80281
v5	$r = 4$, $\alpha = 8$, 5 epochs	0.7729	0.79476

Table 4: Train-val is 4-bit eval (training-time); LB uses bf16 + smart-TTA. v1 LB without TTA: 0.76458.

5.2 TTA Strategy (No Retraining)

Measured on val with the v1 adapter ($n = 1048$):

Strategy	Val acc	Δ
No TTA ($K = 1$)	0.7290	—
Always-TTA ($K = 4$, all)	0.7405	+1.15 pp
Smart-TTA ($K = 4$, skip < 3)	**0.7481**	**+1.91 pp**

Table 5: Smart-TTA dominates; we use it for all submissions starting from v1+sTTA.

Per-bucket smart-TTA breakdown: 2-choice 0.7992 (bypassed, identical to no-TTA), 3-choice 0.7106 (+1.18 pp), 4-choice 0.8492 (+2.78 pp), 5-choice 0.3182 (+4.55 pp). Position bias is largest where invalid letter slots are most numerous — TTA's logit averaging cancels exactly that bias.

5.3 Image Resolution (v2)

The hypothesis was that ScienceQA's diagram-heavy questions would benefit from higher visual token resolution. Empirically, train-val *fell* by 0.48 pp (0.7634 → 0.7586) while LB *rose* by 0.20 pp (0.77665 → 0.77867); both deltas within one standard error. We report this as **statistically tied with baseline**. Plausible reasons: ScienceQA images are mostly < 512 px natively (upsampling adds no information); SmolVLM was pretrained at specific image scales (512-px tiles are mildly OOD); at our parameter budget, visual fidelity is not the binding constraint.

5.4 LoRA Rank (v5) and Decomposition

The capacity-halving control (v5: $r = 4$, $\alpha = 8$, 5 epochs, ~ 2.17M trainable params) yields **0.79476** LB — between v1 and v3. Crossing v1, v3, v5 isolates the two axes:

Comparison	Held fixed	Δ LB
$r = 4 \rightarrow r = 8$	epochs=5	+1.81 pp (v5→ v3)
3 → 5 epochs	$r = 8$	+3.62 pp (v1→ v3)

Table 6: Two-axis decomposition. Training duration is roughly 2 × the lever of rank.

$r = 8$ was the right rank choice (halving costs 1.81 pp), and at our parameter budget and dataset size, fitting time is a stronger constraint than fitting capacity — consistent with the small-data regime (3,109 examples × 3 epochs ≈ 9,300 example-views).

5.5 Training Duration: The Convergence Curve

The single most impactful axis. Three measurements at epochs $\in \{3, 5, 7\}$ trace the generalization curve cleanly:

Epochs	Run	Train-val	LB
3	v1	0.7634	0.77665
5	**v3 (best)**	**0.8015**	**0.81287**
7	v4	0.7996	0.80281

Table 7: Convergence curve at $r = 8$. Both metrics peak at 5 epochs and decline at 7.

The diagnostic that motivated the v3 (3 → 5 epochs) ablation was v1′s training loss — *still descending* at step 580 (final step of epoch 3, loss 0.2535), with no plateau across the descent (1.386 → 0.674 → 0.519 → 0.323 → 0.254). We interpreted this as *under-training*: 9,300 example-views was insufficient. v3′s +3.62 pp LB lift confirmed the diagnosis.

The v4 result (5 → 7 epochs) is a textbook overfitting signature on small data: both train-val and LB decline, but *asymmetrically* (−0.19 pp on train-val vs −1.01 pp on LB) — additional updates fit training-specific patterns that transfer partially to similarly-distributed val but less to held-out test. Across the explored range, training duration affects LB by ~ 5 pp — larger than both rank (~ 1.8 pp) and visual resolution (~ 0 pp).

6 Discussion

6.1 The 4-bit-vs-bf16 Inference Gap

We trained with the base in 4-bit NF4 and computed training-time val with the same 4-bit weights. The submission notebook runs offline and lacks `bitsandbytes`, so it loads the base in `bf16`; the LoRA adapter, optimized assuming 4-bit-dequantized weights, is then applied to the original `bf16` weights. On v1 we measured this gap explicitly: train-val (4-bit) = 0.7634 vs inference val (bf16, no TTA) = 0.7290 — a **3.44 pp drop** from precision mismatch, larger than the < 1 pp typically reported in the QLoRA paper. Our config (LoRA on all 7 projections, including high-magnitude MLP weights) is plausibly more sensitive to dequantization noise than attention-only LoRA. We did not pursue 4-bit inference because bundling `bitsandbytes` wheels into a Kaggle Dataset was disproportionate engineering risk for the expected 1–3 pp lift.

6.2 Val-vs-Leaderboard Ranking Flips and Methodology

The v1 → v2 comparison surfaced a striking val-vs-LB ranking inversion: v2 underperformed v1 on val (−0.48 pp) but outperformed on LB (+0.20 pp). Both deltas are within one standard error; the configurations are **statistically tied** — which itself argued against allocating capacity toward visual resolution and *for* training duration. The v3 result confirmed this hypothesis. More broadly, our methodology was **data-curve-driven**: v1′s loss curve, still descending at step 580, was the empirical signal that motivated v3 (under-trained?). v3 motivated v4 (does more help?). v3 motivated v5 (could we use half the params?). Each ablation tested a hypothesis derived from prior measurements, not a hyperparameter sweep.

7 Reproducibility

Each of the five reported runs (v1–v5) is registered as a named profile in `config.ABLATIONS`. A single command reproduces any row of Table 4:

```
python train.py --ablation v3_epochs5
```

The profile maps to the exact `lora_r`, `lora_alpha`, `epochs`, `img_size`, and `run_name` used; each completed run writes `val_results.json` with the resolved config and val accuracy, pairing recipe with measurement. Submission scoring at `make_submission.py --tta_k 4 --tta_skip_below 3` reproduces the smart-TTA configuration of the reported LB scores. Documented landmines (handled in code): the competition zip's duplicated `images/images/` directory (via `data.stage_competition_data()`), the `transformers v5` removal of `AutoModelForVision2Seq` (via `try/except` import fallback), CUDA state corruption between Kaggle cells (via `empty_cache + synchronize` before `set_seed`), and the QLoRA loss explosion (via the three fixes in Table 2).

8 Conclusion

Under a fixed model, a 5M-trainable-parameter cap, and offline evaluation, we achieved **0.81287** on the public leaderboard via QLoRA at rank 8 across all seven text-decoder linear projections with 5 epochs of training and smart choice-permutation TTA. Training duration was the dominant lever (3 → 5: +3.62 pp; 5 → 7: −1.01 pp), bounding the optimum near 5 epochs. Higher visual resolution was statistically tied with baseline and halving the rank cost 1.81 pp, indicating that fitting time on the small training set — not capacity — was the binding constraint. The 4-bit-vs-`bf16` precision gap forced by the offline-eval rule accounts for ~ 3.4 pp of headroom otherwise available. All measurements use a single random seed (SE ≈ 1.3 pp); an adapter logit-average ensemble across v1–v3 was not pursued and might still extract 0.3–0.7 pp.